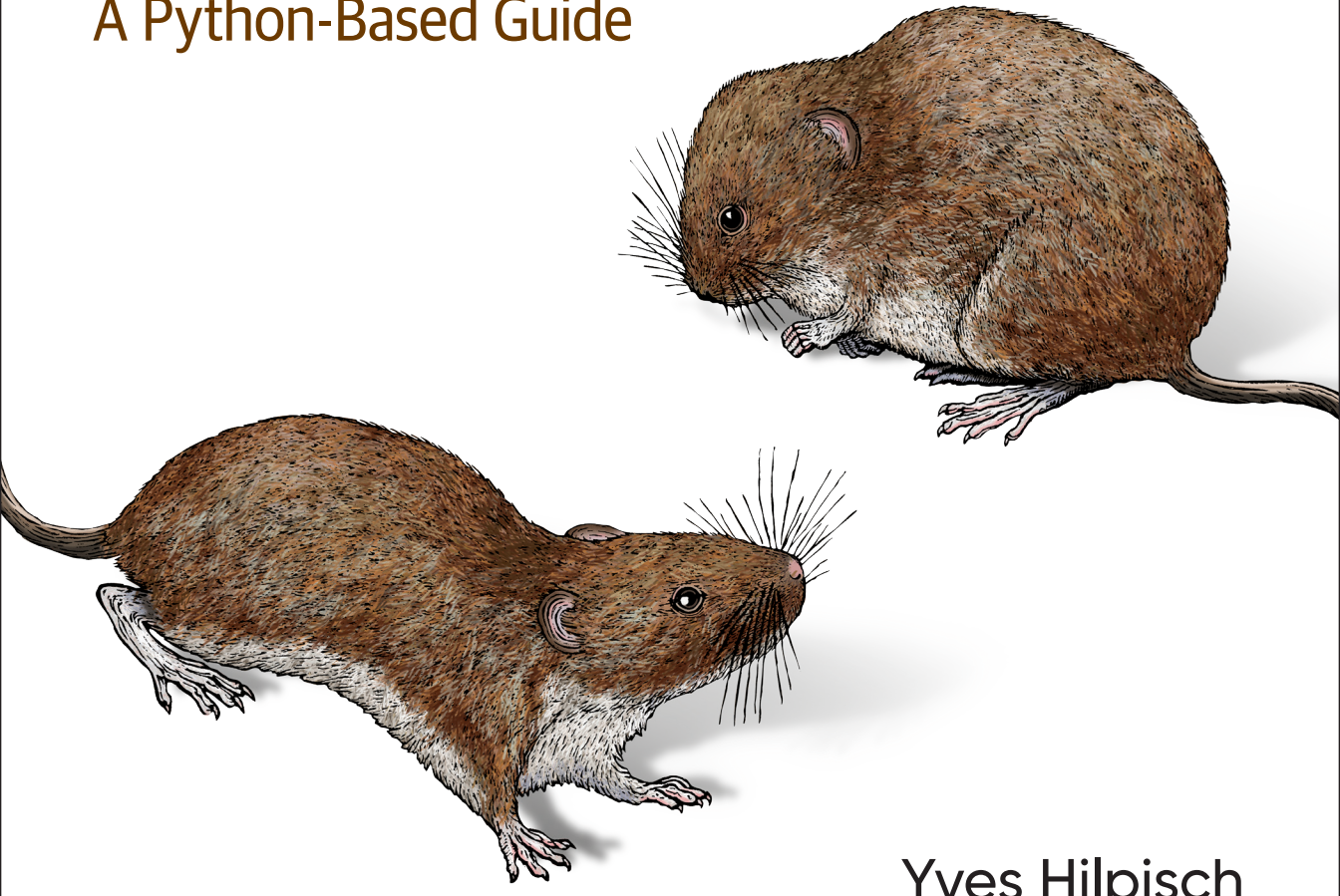
O'REILLY®

Artificial Intelligence in Finance

A Python-Based Guide



Yves Hilpisch

O'REILLY®

Artificial Intelligence in Finance

The widespread adoption of AI and machine learning is revolutionizing many industries today. Once these technologies are combined with the programmatic availability of historical and real-time financial data, the financial industry will also change fundamentally. With this practical book, you'll learn how to use AI and machine learning to discover statistical inefficiencies in financial markets and exploit them through algorithmic trading.

Author Yves Hilpisch shows practitioners, students, and academics in both finance and data science practical ways to apply machine learning and deep learning algorithms to finance. Thanks to lots of self-contained Python examples, you'll be able to replicate all results and figures presented in the book.

In five parts, this guide helps you:

- Learn central notions and algorithms from AI, including recent breakthroughs on the way to artificial general intelligence (AGI) and superintelligence (SI)
- Understand why data-driven finance, AI, and machine learning will have a lasting impact on financial theory and practice
- Apply neural networks and reinforcement learning to discover statistical inefficiencies in financial markets
- Identify and exploit economic inefficiencies through backtesting and algorithmic trading—the automated execution of trading strategies
- Understand how AI will influence the competitive dynamics in the financial industry and what the potential emergence of a financial singularity might bring about

"With its comprehensive and intuitive approach, this will be the main reference book for finance practitioners and academics."

—**Abdullah Karasan**
Data Scientist and Consultant

"An excellent guide for machine learning practitioners looking to tackle a range of problems in quantitative finance."

—**Tim Nugent**
Senior Research Scientist at Refinitiv

Dr. Yves J. Hilpisch is founder and CEO of both The AI Machine and The Python Quants, a group that focuses on the use of open source technologies for financial data science, artificial intelligence, algorithmic trading, and computational finance. Author of *Python for Finance* (O'Reilly), *Python for Algorithmic Trading* (O'Reilly), *Derivatives Analytics with Python* (Wiley), and *Listed Volatility and Variance Derivatives* (Wiley), Yves lectures on AI-powered algorithmic trading in the Certificate in Quantitative Finance (CQF) program.

AI / PYTHON / FINANCE

US $79.99 CAN $105.99
ISBN: 978-1-492-05543-3



Twitter: @oreillymedia
facebook.com/oreilly

Artificial Intelligence in Finance

A Python-Based Guide

Yves Hilpisch

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Artificial Intelligence in Finance
by Yves Hilpisch

Copyright © 2021 Yves Hilpisch. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Michelle Smith
Development Editor: Corbin Collins
Production Editor: Daniel Elfanbaum
Copyeditor: Piper Editorial, LLC
Proofreader: JM Olejarz

Indexer: Potomac Indexing, LLC
Interior Designer: David Futato
Cover Designer: Karen Montgomery
Illustrator: O'Reilly Media, Inc.

October 2020: First Edition

Revision History for the First Edition
2020-10-14: First Release
2020-12-16: Second Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781492055433* for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. Artificial Intelligence in Finance, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights. This book is not intended as financial advice. Please consult a qualified professional if you require financial advice.

978-1-492-05543-3

[LSI]

Table of Contents

Preface ix

Part I. Machine Intelligence

1. Artificial Intelligence 3
Algorithms 3
Types of Data 4
Types of Learning 4
Types of Tasks 8
Types of Approaches 8
Neural Networks 9
OLS Regression 9
Estimation with Neural Networks 13
Classification with Neural Networks 20
Importance of Data 22
Small Data Set 23
Larger Data Set 26
Big Data 28
Conclusions 29
References 30

2. Superintelligence 31
Success Stories 32
Atari 32
Go 38
Chess 40
Importance of Hardware 42

Forms of Intelligence 44
Paths to Superintelligence 45
Networks and Organizations 46
Biological Enhancements 46
Brain-Machine Hybrids 47
Whole Brain Emulation 48
Artificial Intelligence 49
Intelligence Explosion 50
Goals and Control 50
Superintelligence and Goals 51
Superintelligence and Control 53
Potential Outcomes 54
Conclusions 56
References 56

Part II. Finance and Machine Learning

3. Normative Finance. 61
Uncertainty and Risk 62
Definitions 62
Numerical Example 63
Expected Utility Theory 66
Assumptions and Results 66
Numerical Example 69
Mean-Variance Portfolio Theory 72
Assumptions and Results 72
Numerical Example 75
Capital Asset Pricing Model 82
Assumptions and Results 83
Numerical Example 85
Arbitrage Pricing Theory 90
Assumptions and Results 91
Numerical Example 93
Conclusions 95
References 96

4. Data-Driven Finance. 99
Scientific Method 100
Financial Econometrics and Regression 101
Data Availability 104
Programmatic APIs 105

Structured Historical Data 105
Structured Streaming Data 108
Unstructured Historical Data 110
Unstructured Streaming Data 112
Alternative Data 113
Normative Theories Revisited 117
Expected Utility and Reality 118
Mean-Variance Portfolio Theory 123
Capital Asset Pricing Model 130
Arbitrage Pricing Theory 134
Debunking Central Assumptions 143
Normally Distributed Returns 143
Linear Relationships 153
Conclusions 155
References 156
Python Code 156

5. Machine Learning. 161
Learning 162
Data 162
Success 165
Capacity 169
Evaluation 172
Bias and Variance 178
Cross-Validation 180
Conclusions 183
References 183

6. AI-First Finance. 185
Efficient Markets 186
Market Prediction Based on Returns Data 192
Market Prediction with More Features 199
Market Prediction Intraday 204
Conclusions 205
References 207

Part III. Statistical Inefficiencies

7. Dense Neural Networks. 211
The Data 212
Baseline Prediction 214

Normalization 218
Dropout 220
Regularization 222
Bagging 225
Optimizers 227
Conclusions 228
References 228

8. Recurrent Neural Networks. 229
First Example 230
Second Example 234
Financial Price Series 237
Financial Return Series 240
Financial Features 242
Estimation 243
Classification 244
Deep RNNs 245
Conclusions 246
References 247

9. Reinforcement Learning. 249
Fundamental Notions 250
OpenAI Gym 251
Monte Carlo Agent 255
Neural Network Agent 257
DQL Agent 260
Simple Finance Gym 264
Better Finance Gym 268
FQL Agent 271
Conclusions 277
References 278

Part IV. Algorithmic Trading

10. Vectorized Backtesting. 281
Backtesting an SMA-Based Strategy 282
Backtesting a Daily DNN-Based Strategy 289
Backtesting an Intraday DNN-Based Strategy 295
Conclusions 301
References 301

11. Risk Management. 303
Trading Bot 304
Vectorized Backtesting 308
Event-Based Backtesting 311
Assessing Risk 318
Backtesting Risk Measures 322
Stop Loss 324
Trailing Stop Loss 326
Take Profit 328
Conclusions 332
References 332
Python Code 333
Finance Environment 333
Trading Bot 335
Backtesting Base Class 339
Backtesting Class 342

12. Execution and Deployment. 345
Oanda Account 346
Data Retrieval 347
Order Execution 351
Trading Bot 357
Deployment 364
Conclusions 368
References 369
Python Code 369
Oanda Environment 369
Vectorized Backtesting 372
Oanda Trading Bot 373

Part V. Outlook

13. AI-Based Competition. 379
AI and Finance 380
Lack of Standardization 382
Education and Training 383
Fight for Resources 385
Market Impact 386
Competitive Scenarios 387
Risks, Regulation, and Oversight 388

Conclusions 391
References 392

14. Financial Singularity. 395
Notions and Definitions 396
What Is at Stake? 396
Paths to Financial Singularity 400
Orthogonal Skills and Resources 401
Scenarios Before and After 402
Star Trek or Star Wars 403
Conclusions 404
References 404

Part VI. Appendixes

A. Interactive Neural Networks. 407

B. Neural Network Classes. 425

C. Convolutional Neural Networks. 439

Index. 447

Preface

> Will alpha eventually go to zero for every imaginable investment strategy? More fundamentally, is the day approaching when, thanks to so many smart people and smarter computers, financial markets really do become perfect, and we can just sit back, relax, and assume that all assets are priced correctly?
>
> —Robert Shiller (2015)

Artificial intelligence (AI) rose to become a key technology in the 2010s and is assumed to be the dominating technology in the 2020s. Spurred by technological innovations, algorithmic breakthroughs, availability of big data, and ever-increasing compute power, many industries are undergoing fundamental changes driven by AI.

While media and public attention mostly focus on breakthroughs in areas such as gaming and self-driving cars, AI has also become a major technological force in the financial industry. However, it is safe to say that AI in finance is still at a nascent stage —as compared, for example, to industries such as web search or social media.

This book sets out to cover a number of important aspects related to AI in finance. AI in finance is already a vast topic, and a single book needs to focus on selected aspects. Therefore, this book covers the basics first (see Part I and Part II). It then zooms in on discovering *statistical inefficiencies* in financial markets by the use of AI and, more specifically, neural networks (see Part III). Such inefficiencies—embodied by AI algorithms that successfully predict future market movements—are a prerequisite for the exploitation of *economic inefficiencies* through algorithmic trading (see Part IV). Being able to systematically exploit statistical and economic inefficiencies would prove contradictory to one of the established theories and cornerstones in finance: the *efficient market hypothesis* (EMH). The design of a successful *trading bot* can be considered the holy grail in finance to which AI might lead the way. This book concludes by discussing consequences of AI for the financial industry and the possibility of a financial singularity (see Part V). There is also a technical appendix that shows how to build neural networks from scratch based on plain Python code and provides additional examples for their application (see Part VI).

The problem of applying AI to finance is not too dissimilar to the problem of applying AI to other fields. Some major breakthroughs in AI in the 2010s were made possible by the application of reinforcement learning (RL) to playing arcade games, such as those from Atari published in the 1980s (see Mnih et al. 2013), and to board games, such as chess or Go (see Silver et al. 2016). Lessons learned from applying RL in gaming contexts, among other areas, are today applied to such challenging problems as designing and building autonomous vehicles or improving medical diagnostics. Table P-1 compares the application of AI and RL in different domains.

Table P-1. Comparison of AI in different domains

Domain	Agent	Goal	Approach	Reward	Obstacle	Risks
Arcade games	AI agent (software)	Maximizing game score	RL in virtual gaming environment	Points and scores	Planning and delayed rewards	None
Autonomous driving	Self-driving car (software + car)	Safely driving from location A to B	RL in virtual (gaming) environment, real-world test drives	Punishment for mistakes	Transition from virtual to physical world	Damaging property, harming people
Financial trading	Trading bot (software)	Maximizing long-term performance	RL in virtual trading environment	Financial returns	Efficient markets and competition	Financial losses

The beauty of training AI agents to play arcade games lies in the availability of a perfect virtual learning environment[1] and the absence of any kind of risk. With autonomous vehicles, the major problem arises when transitioning from virtual learning environments—for example, a computer game such as *Grand Theft Auto*—to the physical world with a self-driving car navigating real streets populated by other cars and people. This leads to serious risks such as a car causing accidents or harming people.

For a trading bot, RL can also be completely virtual, that is, in a simulated financial market environment. The major risks that arise from malfunctioning trading bots are financial losses and, on an aggregated level, potential systematic risks due to herding by trading bots. Overall, however, the financial domain seems like an ideal place to train, test, and deploy AI algorithms.

Given the rapid developments in the field, it should even be possible for an interested and ambitious student, equipped with a notebook and internet access, to successfully apply AI in a financial trading context. Beyond the hardware and software improvements over recent years, this is due primarily to the rise of online brokers that supply

1 See the Arcade Learning Environment.

historical and real-time financial data and that allow the execution of financial trades via programmatic APIs.

The book is structured in the following six parts.

Part I

The first part discusses central notions and algorithms of AI in general, such as supervised learning and neural networks (see Chapter 1). It also discusses the concept of superintelligence, which relates to an AI agent that possesses human-level intelligence and, in some domains, superhuman-level intelligence (see Chapter 2). Not every researcher in AI believes that superintelligence is possible in the foreseeable future. However, the discussion of this idea provides a valuable framework for discussing AI in general and AI for finance in particular.

Part II

The second part consists of four chapters and is about traditional, normative finance theory (see Chapter 3) and how the field is transformed by data-driven finance (see Chapter 4) and machine learning (ML) (see Chapter 5). Taken together, data-driven finance and ML give rise to a model-free, AI-first approach to finance, as discussed in Chapter 6.

Part III

The third part is about discovering statistical inefficiencies in financial markets by applying deep learning, neural networks, and reinforcement learning. The part covers dense neural networks (DNNs, see Chapter 7), recurrent neural networks (RNNs, see Chapter 8), and algorithms from reinforcement learning (RL, see Chapter 9) that in turn often rely on DNNs to represent and approximate the optimal policy of the AI agent.

Part IV

The fourth part discusses how to exploit statistical inefficiencies through algorithmic trading. Topics are vectorized backtesting (see Chapter 10), event-based backtesting and risk management (see Chapter 11), and execution and deployment of AI-powered algorithmic trading strategies (see Chapter 12).

Part V

The fifth part is about the consequences that arise from AI-based competition in the financial industry (see Chapter 13). It also discusses the possibility of a financial singularity, a point in time at which AI agents would dominate all aspects of finance as we know it. The discussion in this context focuses on artificial financial intelligences as trading bots that consistently generate trading profits above any human or institutional benchmark (see Chapter 14).

Part VI

The Appendix contains Python code for interactive neural network training (see Appendix A), classes for simple and shallow neural networks that are implemented from scratch based on plain Python code (see Appendix B), and an example of how to use convolutional neural networks (CNNs) for financial time series prediction (see Appendix C).

Author's Note

The application of AI to financial trading is still a nascent field, although at the time of writing there are a number of other books available that cover this topic to some extent. Many of these publications, however, fail to show what it means to *economically* exploit statistical inefficiencies.

Some hedge funds already claim to exclusively rely on machine learning to manage their investors' capital. A prominent example is The Voleon Group, a hedge fund that reported more than $6 billion in assets under management at the end of 2019 (see Lee and Karsh 2020). The difficulty of relying on machine learning to outsmart the financial markets is reflected in the fund's performance of 7% for 2019, a year during which the S&P 500 stock index rose by almost 30%.

This book is based on years of practical experience in developing, backtesting, and deploying AI-powered algorithmic trading strategies. The approaches and examples presented are mostly based on my own research since the field is, by nature, not only nascent, but also rather secretive. The exposition and the style throughout this book are relentlessly practical, and in many instances the concrete examples are lacking proper theoretical support and/or comprehensive empirical evidence. This book even presents some applications and examples that might be vehemently criticized by experts in finance and/or machine learning.

For example, some experts in machine and deep learning, such as François Chollet (2017), outright doubt that prediction in financial markets is possible. Certain experts in finance, such as Robert Shiller (2015), doubt that there will ever be something like a financial singularity. Others active at the intersection of the two domains, such as Marcos López de Prado (2018), argue that the use of machine learning for financial trading and investing requires an industrial-scale effort with large teams and huge budgets.

This book does not try to provide a balanced view of or a comprehensive set of references for all the topics covered. The presentation is driven by the personal opinions and experiences of the author, as well as by practical considerations when providing concrete examples and Python code. Many of the examples are also chosen and tweaked to drive home certain points or to show encouraging results. Therefore, it can certainly be argued that results from many examples presented in the book suffer from data snooping and overfitting (for a discussion of these topics, see Hilpisch 2020, ch. 4).

The major goal of this book is to empower the reader to use the code examples in the book as a framework to explore the exciting space of AI applied to financial trading. To achieve this goal, the book relies throughout on a number of simplifying assumptions and primarily on *financial time series data and features derived directly from such data*. In practical applications, a restriction to financial time series data is of course not necessary—a great variety of other types of data and data sources could be used as well. This book's approach to deriving features implicitly assumes that financial time series and features derived from them show *patterns* that, at least to some extent, persist over time and that can be used to predict the direction of future price movements.

Against this background, all examples and code presented in this book are technical and illustrative in nature and do not represent any recommendation or investment advice.

For those who want to deploy approaches and algorithmic trading strategies presented in this book, my book *Python for Algorithmic Trading: From Idea to Cloud Deployment* (O'Reilly) provides more process-oriented and technical details. The two books complement each other in many respects. For readers who are just getting started with Python for finance or who are seeking a refresher and reference manual, my book *Python for Finance: Mastering Data-Driven Finance* (O'Reilly) covers a comprehensive set of important topics and fundamental skills in Python as applied to the financial domain.

References

Papers and books cited in the preface:

Chollet, François. 2017. *Deep Learning with Python*. Shelter Island: Manning.

Hilpisch, Yves. 2018. *Python for Finance: Mastering Data-Driven Finance.* 2nd ed. Sebastopol: O'Reilly.

———. 2020. *Python for Algorithmic Trading: From Idea to Cloud Deployment.* Sebastopol: O'Reilly.

Lee, Justina and Melissa Karsh. 2020. "Machine-Learning Hedge Fund Voleon Group Returns 7% in 2019." *Bloomberg*, January 21, 2020. *https://oreil.ly/TOQiv*.

López de Prado, Marcos. 2018. *Advances in Financial Machine Learning*. Hoboken, NJ: John Wiley & Sons.

Mnih, Volodymyr et al. 2013. "Playing Atari with Deep Reinforcement Learning." arXiv. December 19. *https://oreil.ly/-pW-1*.

Shiller, Robert. 2015. “The Mirage of the Financial Singularity.” Yale Insights. July 16. *https://oreil.ly/VRkP3*.

Silver, David et al. 2016. “Mastering the Game of Go with Deep Neural Networks and Tree Search.” *Nature* 529 (January): 484-489.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`
: Shows commands or other text that should be typed literally by the user.

`Constant width italic`
: Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates important information.



This element indicates a warning or caution.

Using Code Examples

You can access and execute the code that accompanies the book on the Quant Platform at *https://aiif.pqp.io*, for which only a free registration is required.

If you have a technical question or a problem using the code examples, please send an email to *bookquestions@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example, this book may be attributed as: "*Artificial Intelligence in Finance* by Yves Hilpisch (O'Reilly). Copyright 2021 Yves Hilpisch, 978-1-492-05543-3."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *http://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/ai-in-finance*.

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book.

For news and information about our books and courses, visit *http://oreilly.com*.

Find us on Facebook: *http://facebook.com/oreilly*

Follow us on Twitter: *http://twitter.com/oreillymedia*

Watch us on YouTube: *http://www.youtube.com/oreillymedia*

Acknowledgments

I want to thank the technical reviewers—Margaret Maynard-Reid, Dr. Tim Nugent, and Dr. Abdullah Karasan—who did a great job in helping me improve the contents of the book.

Delegates of the Certificate Programs in Python for Computational Finance and Algorithmic Trading also helped improve this book. Their ongoing feedback has enabled me to weed out errors and mistakes and refine the code and notebooks used in our online training classes and now, finally, in this book.

The same holds true for the team members of The Python Quants and The AI Machine. In particular, Michael Schwed, Ramanathan Ramakrishnamoorthy, and Prem Jebaseelan support me in numerous ways. They are the ones who assist me with the difficult technical problems that arise during the writing of a book like this one.

I would also like to thank the whole team at O'Reilly Media—especially Michelle Smith, Corbin Collins, Victoria DeRose, and Danny Elfanbaum—for making it all happen and helping me refine the book in so many ways.

Of course, all remaining errors are mine alone.

Furthermore, I would also like to thank the team at Refinitiv—in particular, Jason Ramchandani—for providing ongoing support and access to financial data. The major data files used throughout the book and made available to the readers were received in one way or another from Refinitiv's data APIs.

Of course, everybody making use of artificial intelligence and machine learning today benefits from the achievements and contributions of so many others. Therefore, we should always recall what Sir Isaac Newton wrote in 1675: "If I have seen further it is by standing on the shoulders of Giants." In that sense, a big thank you to all the researchers and open source maintainers contributing to the field.

Finally, special thanks go to my family, who support me all year round in my business and book-writing activities. In particular, I thank my wife Sandra for relentlessly taking care of us all and for providing us with a home and environment that we all love so much. I dedicate this book to my lovely wife Sandra and my wonderful son Henry.

PART I
Machine Intelligence

> Today's algorithmic trading programs are relatively simple and make only limited use of AI. This is sure to change.
>
> —Murray Shanahan (2015)

This part is about artificial intelligence (AI) in general: *artificial* in the sense that the intelligence is not displayed by a biological organism but rather by a machine, and *intelligence* as defined by AI researcher Max Tegmark as the "ability to accomplish complex goals." This part introduces central notions and algorithms from the AI field, gives examples of major recent breakthroughs, and discusses aspects of superintelligence. It consists of two chapters:

- Chapter 1 introduces general notions, ideas, and definitions from the field of AI. It also provides several Python examples of how different algorithms can be applied in practice.
- Chapter 2 discusses concepts and topics related to artificial general intelligence (AGI) and superintelligence (SI). These types of intelligence relate to AI agents that have reached at least human-level intelligence in all domains and superhuman intelligence in certain domains.

CHAPTER 1

Artificial Intelligence

> This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.
>
> —David Silver et al. (2016)

This chapter introduces general notions, ideas, and definitions from the field of artificial intelligence (AI) for the purposes of this book. It also provides worked-out examples for different types of major learning algorithms. In particular, "Algorithms" on page 3 takes a broad perspective and categorizes types of data, types of learning, and types of problems typically encountered in an AI context. This chapter also presents examples for unsupervised and reinforcement learning. "Neural Networks" on page 9 jumps right into the world of neural networks, which not only are central to what follows in later chapters of the book but also have proven to be among the most powerful algorithms AI has to offer nowadays. "Importance of Data" on page 22 discusses the importance of data volume and variety in the context of AI.

Algorithms

This section introduces basic notions from the field of AI relevant to this book. It discusses the different types of data, learning, problems, and approaches that can be subsumed under the general term *AI*. Alpaydin (2016) provides an informal introduction to and overview of many of the topics covered only briefly in this section, along with many examples.

Types of Data

Data in general has two major components:

Features
: Features data (or input data) is data that is given as input to an algorithm. In a financial context, this might be, for example, the income and the savings of a potential debtor.

Labels
: Labels data (or output data) is data that is given as the relevant output to be learned, for example, by a supervised learning algorithm. In a financial context, this might be the creditworthiness of a potential debtor.

Types of Learning

There are three major types of learning algorithms:

Supervised learning (SL)
: These are algorithms that learn from a given sample data set of features (input) and labels (output) values. The next section presents examples for such algorithms, like ordinary least-squares (OLS) regression and neural networks. The purpose of supervised learning is to learn the relationship between the input and output values. In finance, such algorithms might be trained to predict whether a potential debtor is creditworthy or not. For the purposes of this book, these are the most important types of algorithms.

Unsupervised learning (UL)
: These are algorithms that learn from a given sample data set of features (input) values only, often with the goal of finding structure in the data. They are supposed to learn about the input data set, given, for example, some guiding parameters. Clustering algorithms fall into that category. In a financial context, such algorithms might cluster stocks into certain groups.

Reinforcement learning (RL)
: These are algorithms that learn from trial and error by receiving a reward for taking an action. They update an optimal action policy according to what rewards and punishments they receive. Such algorithms are, for example, used for environments where actions need to be taken continuously and rewards are received immediately, such as in a computer game.

Because supervised learning is addressed in the subsequent section in some detail, brief examples will illustrate unsupervised learning and reinforcement learning.

Unsupervised Learning

Simply speaking, a *k-means clustering algorithm* sorts *n* observations into *k* clusters. Each observation belongs to the cluster to which its mean (center) is nearest. The following Python code generates sample data for which the features data is clustered. Figure 1-1 visualizes the clustered sample data and also shows that the `scikit-learn` `KMeans` algorithm used here has identified the clusters perfectly. The coloring of the dots is based on what the algorithm has learned.[1]

```
In [1]: import numpy as np
        import pandas as pd
        from pylab import plt, mpl
        plt.style.use('seaborn')
        mpl.rcParams['savefig.dpi'] = 300
        mpl.rcParams['font.family'] = 'serif'
        np.set_printoptions(precision=4, suppress=True)

In [2]: from sklearn.cluster import KMeans
        from sklearn.datasets import make_blobs

In [3]: x, y = make_blobs(n_samples=100, centers=4,
                          random_state=500, cluster_std=1.25)  ❶

In [4]: model = KMeans(n_clusters=4, random_state=0)  ❷

In [5]: model.fit(x)  ❸
Out[5]: KMeans(n_clusters=4, random_state=0)

In [6]: y_ = model.predict(x)  ❹

In [7]: y_  ❺
Out[7]: array([3, 3, 1, 2, 1, 1, 3, 2, 1, 2, 2, 3, 2, 0, 0, 3, 2, 0, 2, 0, 0, 3,
               1, 2, 1, 1, 0, 0, 1, 3, 2, 1, 1, 0, 1, 3, 1, 3, 2, 2, 2, 1, 0, 0,
               3, 1, 2, 0, 2, 0, 3, 0, 1, 0, 1, 3, 1, 2, 0, 3, 1, 0, 3, 2, 3, 0,
               1, 1, 1, 2, 3, 1, 2, 0, 2, 3, 2, 0, 2, 2, 1, 3, 1, 3, 2, 2, 3, 2,
               0, 0, 0, 3, 3, 3, 3, 0, 3, 1, 0, 0], dtype=int32)

In [8]: plt.figure(figsize=(10, 6))
        plt.scatter(x[:, 0], x[:, 1], c=y_,  cmap='coolwarm');
```

❶ A sample data set is created with clustered features data.

❷ A `KMeans` model object is instantiated, fixing the number of clusters.

❸ The model is fitted to the features data.

1 For details, see `sklearn.cluster.KMeans` and VanderPlas (2017, ch. 5).

❹ The predictions are generated given the fitted model.

❺ The predictions are numbers from 0 to 3, each representing one cluster.

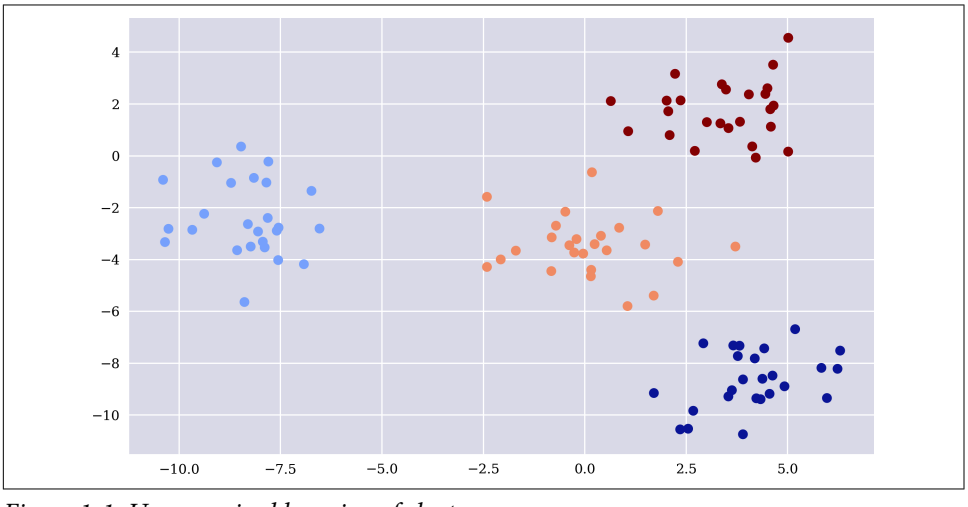


Figure 1-1. Unsupervised learning of clusters

Once an algorithm such as `KMeans` is trained, it can, for instance, predict the cluster for a new (not yet seen) combination of features values. Assume that such an algorithm is trained on features data that describes potential and real debtors of a bank. It might learn about the creditworthiness of potential debtors by generating two clusters. New potential debtors can then be sorted into a certain cluster: "creditworthy" versus "not creditworthy."

Reinforcement learning

The following example is based on a coin tossing game that is played with a coin that lands 80% of the time on heads and 20% of the time on tails. The coin tossing game is heavily biased to emphasize the benefits of learning as compared to an uninformed baseline algorithm. The baseline algorithm, which bets randomly and equally distributes on heads and tails, achieves a total reward of around 50, on average, per epoch of 100 bets played:

```
In [9]: ssp = [1, 1, 1, 1, 0]  ❶

In [10]: asp = [1, 0]  ❷

In [11]: def epoch():
             tr = 0
             for _ in range(100):
                 a = np.random.choice(asp)  ❸
                 s = np.random.choice(ssp)  ❹
```

```
                if a == s:
                    tr += 1  ❺
            return tr

In [12]: rl = np.array([epoch() for _ in range(15)])  ❻
         rl
Out[12]: array([53, 55, 50, 48, 46, 41, 51, 49, 50, 52, 46, 47, 43, 51, 52])

In [13]: rl.mean()  ❼
Out[13]: 48.93333333333333
```

❶ The state space (1 = heads, 0 = tails).

❷ The action space (1 = bet on heads, 0 = bet on tails).

❸ An action is randomly chosen from the action space.

❹ A state is randomly chosen from the state space.

❺ The total reward `tr` is increased by one if the bet is correct.

❻ The game is played for a number of epochs; each epoch is 100 bets.

❼ The average total reward of the epochs played is calculated.

Reinforcement learning tries to learn from what is observed after an action is taken, usually based on a reward. To keep things simple, the following learning algorithm only keeps track of the states that are observed in each round insofar as they are appended to the action space `list` object. In this way, the algorithm learns the bias in the game, though maybe not perfectly. By randomly sampling from the updated action space, the bias is reflected because naturally the bet will more often be heads. Over time, heads is chosen, on average, around 80% of the time. The average total reward of around 65 reflects the improvement of the learning algorithm as compared to the uninformed baseline algorithm:

```
In [14]: ssp = [1, 1, 1, 1, 0]

In [15]: def epoch():
             tr = 0
             asp = [0, 1]  ❶
             for _ in range(100):
                 a = np.random.choice(asp)
                 s = np.random.choice(ssp)
                 if a == s:
                     tr += 1
                 asp.append(s)  ❷
             return tr
```

```
In [16]: rl = np.array([epoch() for _ in range(15)])
         rl
Out[16]: array([64, 65, 77, 65, 54, 64, 71, 64, 57, 62, 69, 63, 61, 66, 75])

In [17]: rl.mean()
Out[17]: 65.13333333333334
```

❶ Resets the action space before starting (over)

❷ Adds the observed state to the action space

Types of Tasks

Depending on the type of labels data and the problem at hand, two types of tasks to be learned are important:

Estimation
: Estimation (or approximation, regression) refers to the cases in which the labels data is real-valued (continuous); that is, it is technically represented as floating point numbers.

Classification
: Classification refers to the cases in which the labels data consists of a finite number of classes or categories that are typically represented by discrete values (positive natural numbers), which in turn are represented technically as integers.

The following section provides examples for both types of tasks.

Types of Approaches

Some more definitions might be in order before finishing this section. This book follows the common differentiation between the following three major terms:

Artificial intelligence (AI)
: AI encompasses all types of learning (algorithms), as defined before, and some more (for example, expert systems).

Machine learning (ML)
: ML is the discipline of learning relationships and other information about given data sets based on an algorithm and a measure of success; a measure of success might, for example, be the mean-squared error (MSE) given labels values and output values to be estimated and the predicted values from the algorithm. ML is a sub-set of AI.

Deep learning (DL)
: DL encompasses all algorithms based on neural networks. The term *deep* is usually only used when the neural network has more than one hidden layer. DL is a sub-set of machine learning and so is therefore also a sub-set of AI.

DL has proven useful for a number of broad problem areas. It is suited for estimation and classification tasks, as well as for RL. In many cases, DL-based approaches perform better than alternative algorithms, such as logistic regression or kernel-based ones, like support vector machines.[2] That is why this book mainly focuses on DL. DL approaches used include dense neural networks (DNNs), recurrent neural networks (RNNs), and convolutional neural networks (CNNs). More details appear in later chapters, particularly in Part III.

Neural Networks

The previous sections provide a broader overview of algorithms in AI. This section shows how neural networks fit in. A simple example will illustrate what characterizes neural networks in comparison to traditional statistical methods, such as ordinary least-squares (OLS) regression. The example starts with mathematics and then uses linear regression for *estimation* (or function approximation) and finally applies neural networks to accomplish the estimation. The approach taken here is a supervised learning approach where the task is to estimate labels data based on features data. This section also illustrates the use of neural networks in the context of *classification* problems.

OLS Regression

Assume that a mathematical function is given as follows:

$$f:\mathbb{R} \rightarrow \mathbb{R}, y = 2x^2 - \frac{1}{3}x^3$$

Such a function transforms an input value x to an output value y. Or it transforms a series of input values $x_1, x_2, ..., x_N$ into a series of output values $y_1, y_2, ..., y_N$. The following Python code implements the mathematical function as a Python function and creates a number of input and output values. Figure 1-2 plots the output values against the input values:

```
In [18]: def f(x):
             return 2 * x ** 2 - x ** 3 / 3  ❶
```

2 For details, see VanderPlas (2017, ch. 5).

```
In [19]: x = np.linspace(-2, 4, 25)  ❷
         x  ❷
Out[19]: array([-2.  , -1.75, -1.5 , -1.25, -1.  , -0.75, -0.5 , -0.25,  0.  ,
                 0.25,  0.5 ,  0.75,  1.  ,  1.25,  1.5 ,  1.75,  2.  ,  2.25,
                 2.5 ,  2.75,  3.  ,  3.25,  3.5 ,  3.75,  4.  ])

In [20]: y = f(x)  ❸
         y  ❸
Out[20]: array([10.6667,  7.9115,  5.625 ,  3.776 ,  2.3333,  1.2656,  0.5417,
                 0.1302,  0.    ,  0.1198,  0.4583,  0.9844,  1.6667,  2.474 ,
                 3.375 ,  4.3385,  5.3333,  6.3281,  7.2917,  8.1927,  9.    ,
                 9.6823, 10.2083, 10.5469, 10.6667])

In [21]: plt.figure(figsize=(10, 6))
         plt.plot(x, y, 'ro');
```

❶ The mathematical function as a Python function

❷ The input values

❸ The output values

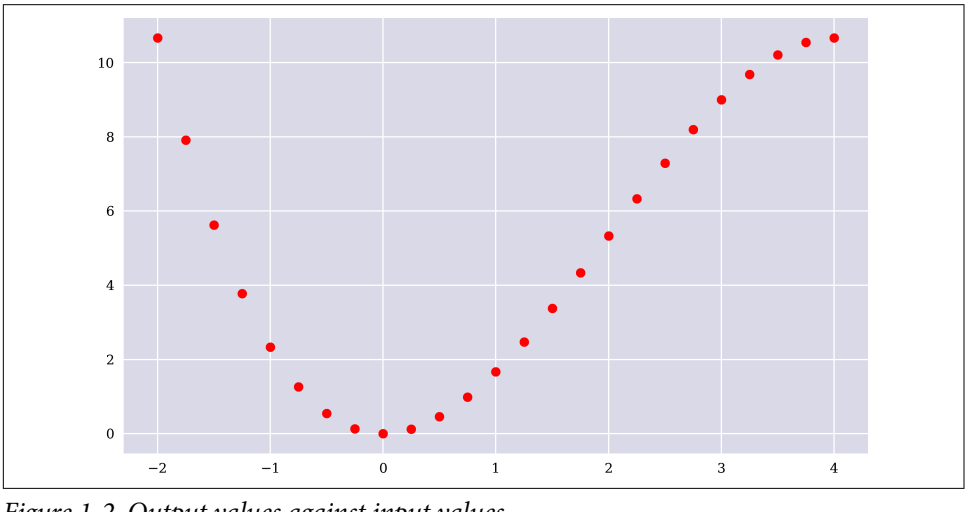


Figure 1-2. Output values against input values

Whereas in the mathematical example the function comes first, the input data second, and the output data third, the sequence is different in *statistical learning*. Assume that the previous input values and output values are given. They represent the *sample* (data). The problem in *statistical regression* is to find a function that approximates the functional relationship between the input values (also called the *independent values*) and the output values (also called the *dependent values*) as well as possible.

Assume simple OLS linear regression. In this case, the functional relationship between the input and output values is assumed to be linear, and the problem is to find optimal parameters α and β for the following linear equation:

$$\hat{f}: \mathbb{R} \rightarrow \mathbb{R}, \hat{y} = \alpha + \beta x$$

For given input values $x_1, x_2, ..., x_N$ and output values $y_1, y_2, ..., y_N$, *optimal* in this case means that they minimize the mean squared error (MSE) between the real output values and the approximated output values:

$$\min_{\alpha,\beta} \frac{1}{N} \sum_{n}^{N} \left(y_n - \hat{f}(x_n) \right)^2$$

For the case of simple linear regression, the solution (α^*, β^*) is known in closed form, as shown in the following equation. Bars on the variables indicate sample mean values:

$$\begin{aligned} \beta^* &= \frac{Cov(x, y)}{Var(x)} \\ \alpha^* &= \bar{y} - \beta \bar{x} \end{aligned}$$

The following Python code calculates the optimal parameter values, linearly estimates (approximates) the output values, and plots the linear regression line alongside the sample data (see Figure 1-3). The linear regression approach does not work too well here in approximating the functional relationship. This is confirmed by the relatively high MSE value:

```
In [22]: beta = np.cov(x, y, ddof=0)[0, 1] / np.var(x)  ❶
         beta  ❶
Out[22]: 1.0541666666666667

In [23]: alpha = y.mean() - beta * x.mean()  ❷
         alpha  ❷
Out[23]: 3.8625000000000003

In [24]: y_ = alpha + beta * x  ❸

In [25]: MSE = ((y - y_) ** 2).mean()  ❹
         MSE  ❹
Out[25]: 10.721953125

In [26]: plt.figure(figsize=(10, 6))
         plt.plot(x, y, 'ro', label='sample data')
         plt.plot(x, y_, lw=3.0, label='linear regression')
         plt.legend();
```

❶ Calculation of optimal β

❷ Calculation of optimal α

❸ Calculation of estimated output values

❹ Calculation of the MSE given the approximation

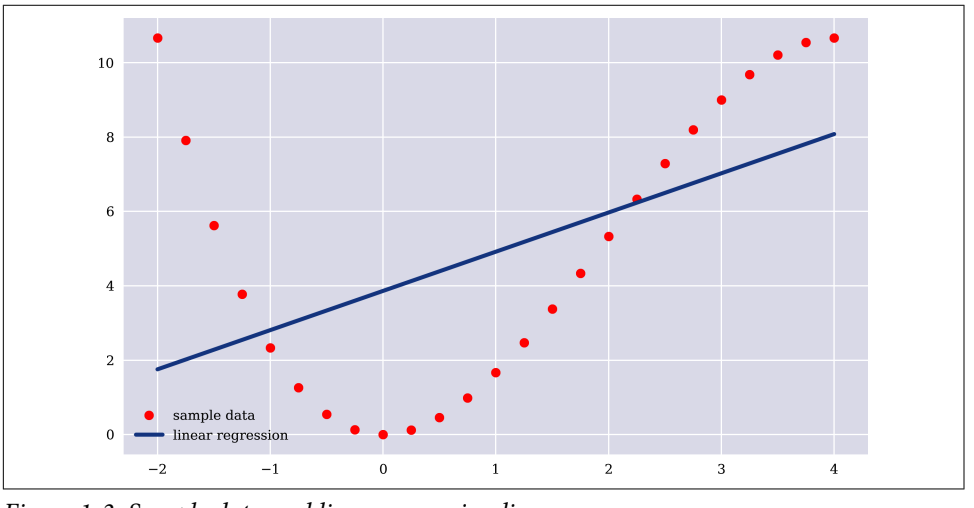


Figure 1-3. Sample data and linear regression line

How can the MSE value be improved (decreased)—maybe even to 0, that is, to a "perfect estimation?" Of course, OLS regression is not constrained to a simple linear relationship. In addition to the constant and linear terms, higher order monomials, for instance, can be easily added as basis functions. To this end, compare the regression results shown in Figure 1-4 and the following code that creates the figure. The improvements that come from using quadratic and cubic monomials as basis functions are obvious and also are numerically confirmed by the calculated MSE values. For basis functions up to and including the cubic monomial, the estimation is perfect, and the functional relationship is perfectly recovered:

```
In [27]: plt.figure(figsize=(10, 6))
         plt.plot(x, y, 'ro', label='sample data')
         for deg in [1, 2, 3]:
             reg = np.polyfit(x, y, deg=deg)  ❶
             y_ = np.polyval(reg, x)  ❷
             MSE = ((y - y_) ** 2).mean()  ❸
             print(f'deg={deg} | MSE={MSE:.5f}')
             plt.plot(x, np.polyval(reg, x), label=f'deg={deg}')
         plt.legend();
         deg=1 | MSE=10.72195
         deg=2 | MSE=2.31258
```

```
        deg=3 | MSE=0.00000

In [28]: reg  ❹
Out[28]: array([-0.3333,  2.    ,  0.    , -0.    ])
```

❶ Regression step

❷ Approximation step

❸ MSE calculation

❹ Optimal ("perfect") parameter values

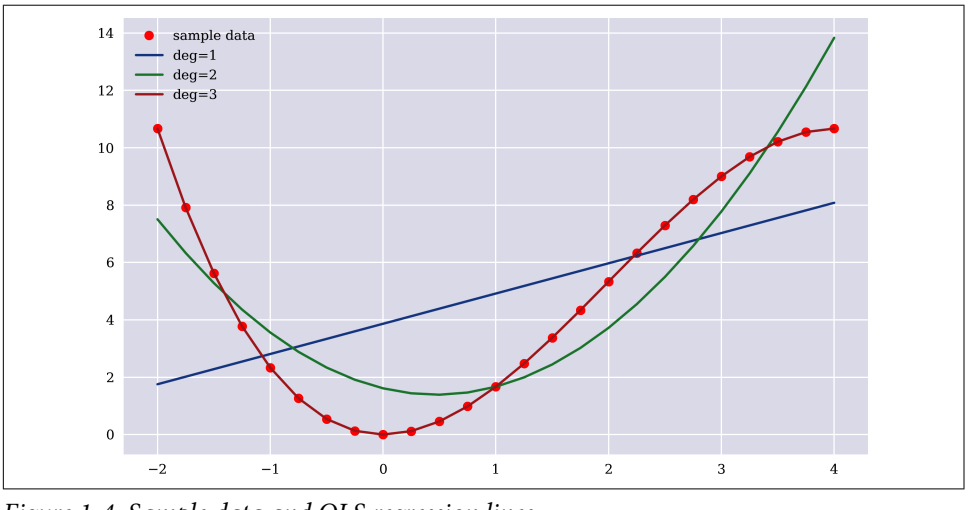


Figure 1-4. Sample data and OLS regression lines

Exploiting the knowledge of the form of the mathematical function to be approximated and accordingly adding more basis functions to the regression leads to a "perfect approximation." That is, the OLS regression recovers the exact factors of the quadratic and cubic part, respectively, of the original function.

Estimation with Neural Networks

However, not all relationships are of this kind. This is where, for instance, *neural networks* can help. Without going into the details, neural networks can approximate a wide range of functional relationships. Knowledge of the form of the relationship is generally not required.

Scikit-learn

The following Python code uses the `MLPRegressor` class of `scikit-learn`, which implements a DNN for estimation. DNNs are sometimes also called multi-layer perceptron (MLP).[3] The results are not perfect, as Figure 1-5 and the MSE illustrate. However, they are quite good already for the simple configuration used:

```
In [29]: from sklearn.neural_network import MLPRegressor

In [30]: model = MLPRegressor(hidden_layer_sizes=3 * [256],
                              learning_rate_init=0.03,
                              max_iter=5000)  ❶

In [31]: model.fit(x.reshape(-1, 1), y)  ❷
Out[31]: MLPRegressor(hidden_layer_sizes=[256, 256, 256], learning_rate_init=0.03,
                      max_iter=5000)

In [32]: y_ = model.predict(x.reshape(-1, 1))  ❸

In [33]: MSE = ((y - y_) ** 2).mean()
         MSE
Out[33]: 0.021662355744355866

In [34]: plt.figure(figsize=(10, 6))
         plt.plot(x, y, 'ro', label='sample data')
         plt.plot(x, y_, lw=3.0, label='dnn estimation')
         plt.legend();
```

❶ Instantiates the `MLPRegressor` object

❷ Implements the fitting or learning step

❸ Implements the prediction step

Just having a look at the results in Figure 1-4 and Figure 1-5, one might assume that the methods and approaches are not too dissimilar after all. However, there is a fundamental difference worth highlighting. Although the OLS regression approach, as shown explicitly for the simple linear regression, is based on the calculation of certain well-specified quantities and parameters, the neural network approach relies on *incremental learning*. This in turn means that a set of parameters, the *weights* within the neural network, are first initialized randomly and then adjusted gradually given the differences between the neural network output and the sample output values. This approach lets you retrain (update) a neural network incrementally.

3 For details, see `sklearn.neural_network.MLPRegressor`. For more background, see Goodfellow et al. (2016, ch. 6).

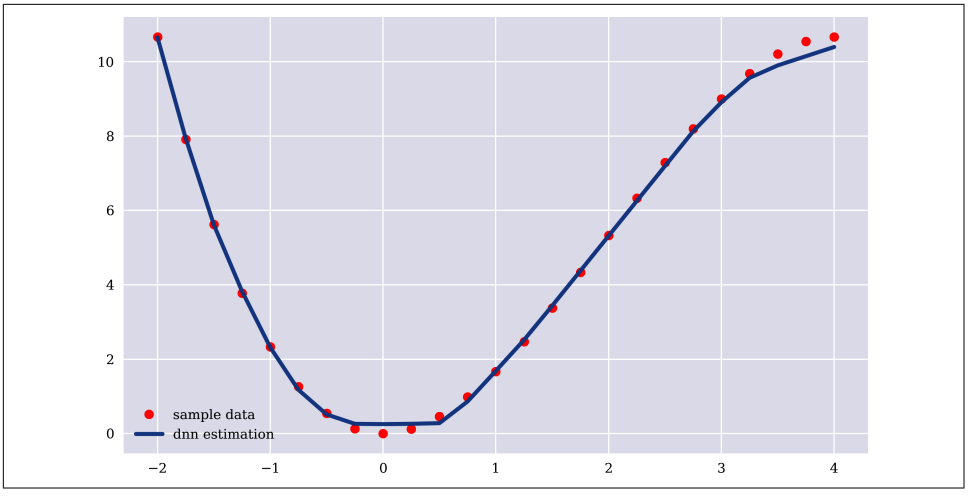


Figure 1-5. Sample data and neural network–based estimations

Keras

The next example uses a sequential model with the `Keras` deep learning package.[4] The model is fitted, or *trained*, for 100 epochs. The procedure is repeated for five rounds. After every such round, the approximation by the neural network is updated and plotted. Figure 1-6 shows how the approximation gradually improves with every round. This is also reflected in the decreasing MSE values. The end result is not perfect, but again, it is quite good given the simplicity of the model:

```
In [35]: import tensorflow as tf
         tf.random.set_seed(100)

In [36]: from keras.layers import Dense
         from keras.models import Sequential
         Using TensorFlow backend.

In [37]: model = Sequential()  ❶
         model.add(Dense(256, activation='relu', input_dim=1))  ❷
         model.add(Dense(1, activation='linear'))  ❸
         model.compile(loss='mse', optimizer='rmsprop')  ❹

In [38]: ((y - y_) ** 2).mean()
Out[38]: 0.021662355744355866

In [39]: plt.figure(figsize=(10, 6))
         plt.plot(x, y, 'ro', label='sample data')
         for _ in range(1, 6):
```

4 For details, see Chollet (2017, ch. 3).

```
    model.fit(x, y, epochs=100, verbose=False)  ❺
    y_ =  model.predict(x)  ❻
    MSE = ((y - y_.flatten()) ** 2).mean()  ❼
    print(f'round={_} | MSE={MSE:.5f}')
    plt.plot(x, y_, '--', label=f'round={_}')  ❽
plt.legend();
round=1 | MSE=3.09714
round=2 | MSE=0.75603
round=3 | MSE=0.22814
round=4 | MSE=0.11861
round=5 | MSE=0.09029
```

❶ Instantiates the `Sequential` model object

❷ Adds a densely connected hidden layer with rectified linear unit (ReLU) activation[5]

❸ Adds the output layer with linear activation

❹ Compiles the model for usage

❺ Trains the neural network for a fixed number of epochs

❻ Implements the approximation step

❼ Calculates the current MSE

❽ Plots the current approximation results

Roughly speaking, one can say that the neural network does almost as well in the estimation as the OLS regression, which delivers a perfect result. Therefore, why use neural networks at all? A more comprehensive answer might need to come later in this book, but a somewhat different example might give some hint.

Consider instead the previous sample data set, as generated from a well-defined mathematical function, now a *random sample data set*, for which both features and labels are randomly chosen. Of course, this example is for illustration and does not allow for a deep interpretation.

5 For details on activation functions with `Keras`, see *https://keras.io/activations*.

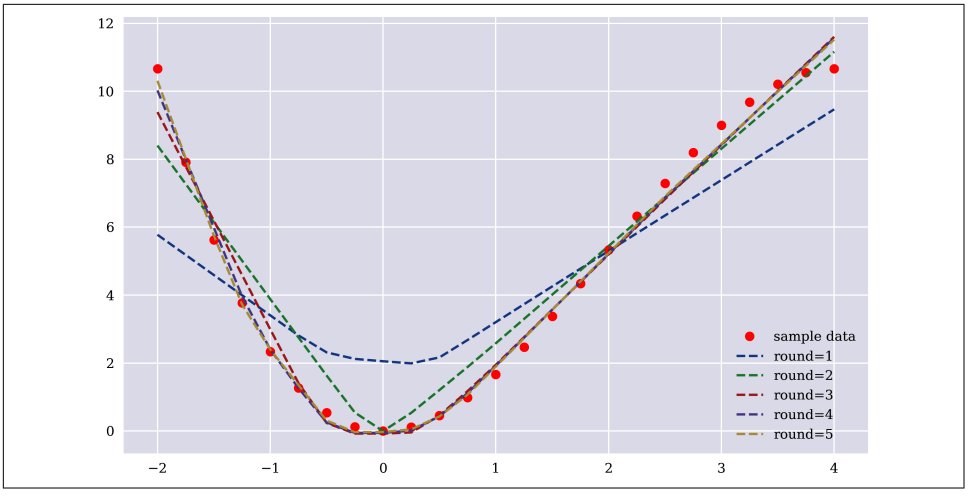


Figure 1-6. Sample data and estimations after multiple training rounds

The following code generates the random sample data set and creates the OLS regression estimation based on a varying number of monomial basis functions. Figure 1-7 visualizes the results. Even for the highest number of monomials in the example, the estimation results are still not too good. The MSE value is accordingly relatively high:

```
In [40]: np.random.seed(0)
         x = np.linspace(-1, 1)
         y = np.random.random(len(x)) * 2 - 1

In [41]: plt.figure(figsize=(10, 6))
         plt.plot(x, y, 'ro', label='sample data')
         for deg in [1, 5, 9, 11, 13, 15]:
             reg = np.polyfit(x, y, deg=deg)
             y_ = np.polyval(reg, x)
             MSE = ((y - y_) ** 2).mean()
             print(f'deg={deg:2d} | MSE={MSE:.5f}')
             plt.plot(x, np.polyval(reg, x), label=f'deg={deg}')
         plt.legend();
         deg= 1 | MSE=0.28153
         deg= 5 | MSE=0.27331
         deg= 9 | MSE=0.25442
         deg=11 | MSE=0.23458
         deg=13 | MSE=0.22989
         deg=15 | MSE=0.21672
```

The results for the OLS regression are not too surprising. OLS regression in this case assumes that the approximation can be achieved through an appropriate combination of a finite number of basis functions. Since the sample data set has been generated randomly, the OLS regression does not perform well in this case.

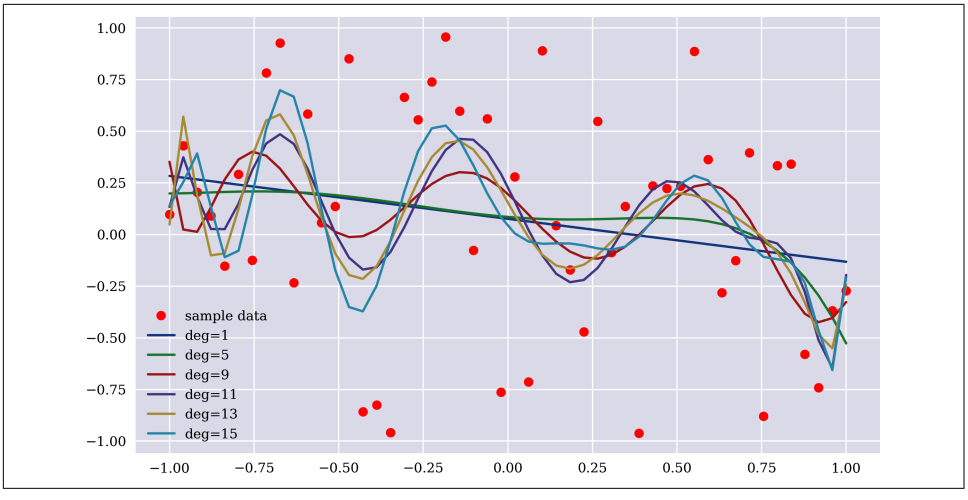


Figure 1-7. Random sample data and OLS regression lines

What about neural networks? The application is as straightforward as before and yields estimations as shown in Figure 1-8. While the end result is not perfect, it is obvious that the neural network performs better than the OLS regression in estimating the random label values from the random features values. Given its architecture, however, the neural network has almost 200,000 trainable parameters (weights), which offers relatively high flexibility, particularly when compared to the OLS regression, for which a maximum of 15 + 1 parameters are used:

```
In [42]: model = Sequential()
         model.add(Dense(256, activation='relu', input_dim=1))
         for _ in range(3):
             model.add(Dense(256, activation='relu'))  ❶
         model.add(Dense(1, activation='linear'))
         model.compile(loss='mse', optimizer='rmsprop')

In [43]: model.summary()  ❷
         Model: "sequential_2"
         _________________________________________________________________
         Layer (type)                 Output Shape              Param #
         =================================================================
         dense_3 (Dense)              (None, 256)               512
         _________________________________________________________________
         dense_4 (Dense)              (None, 256)               65792
         _________________________________________________________________
         dense_5 (Dense)              (None, 256)               65792
         _________________________________________________________________
         dense_6 (Dense)              (None, 256)               65792
         _________________________________________________________________
         dense_7 (Dense)              (None, 1)                 257
         =================================================================
```

```
Total params: 198,145
Trainable params: 198,145
Non-trainable params: 0
_________________________________________________________________
```

```
In [44]: %%time
         plt.figure(figsize=(10, 6))
         plt.plot(x, y, 'ro', label='sample data')
         for _ in range(1, 8):
             model.fit(x, y, epochs=500, verbose=False)
             y_ =  model.predict(x)
             MSE = ((y - y_.flatten()) ** 2).mean()
             print(f'round={_} | MSE={MSE:.5f}')
             plt.plot(x, y_, '--', label=f'round={_}')
         plt.legend();
         round=1 | MSE=0.13560
         round=2 | MSE=0.08337
         round=3 | MSE=0.06281
         round=4 | MSE=0.04419
         round=5 | MSE=0.03329
         round=6 | MSE=0.07676
         round=7 | MSE=0.00431
         CPU times: user 30.4 s, sys: 4.7 s, total: 35.1 s
         Wall time: 13.6 s
```

❶ Multiple hidden layers are added.

❷ Network architecture and number of trainable parameters are shown.

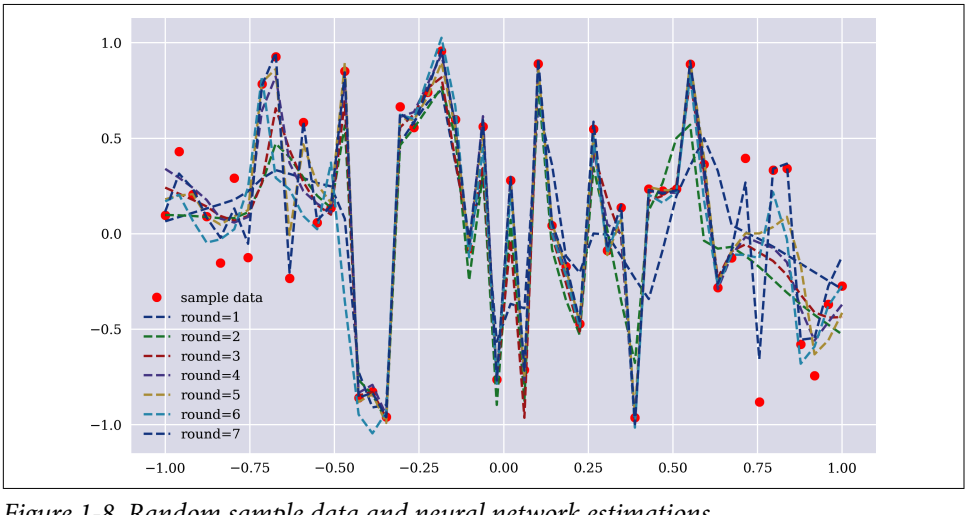


Figure 1-8. Random sample data and neural network estimations

Classification with Neural Networks

Another benefit of neural networks is that they can be easily used for classification tasks as well. Consider the following Python code that implements a classification using a neural network based on `Keras`. The binary features data and labels data are generated randomly. The major adjustment to be made modeling-wise is to change the activation function from the output layer to `sigmoid` from `linear`. More details on this appear in later chapters. The classification is not perfect. However, it reaches a high level of accuracy. How the accuracy, expressed as the relationship between correct results to all label values, changes with the number of training epochs is shown in Figure 1-9. The accuracy starts out low and then improves step-wise, though not necessarily with every step:

```
In [45]: f = 5
         n = 10

In [46]: np.random.seed(100)

In [47]: x = np.random.randint(0, 2, (n, f))  ❶
         x  ❶
Out[47]: array([[0, 0, 1, 1, 1],
                [1, 0, 0, 0, 0],
                [0, 1, 0, 0, 0],
                [0, 1, 0, 0, 1],
                [0, 1, 0, 0, 0],
                [1, 1, 1, 0, 0],
                [1, 0, 0, 1, 1],
                [1, 1, 1, 0, 0],
                [1, 1, 1, 1, 1],
                [1, 1, 1, 0, 1]])

In [48]: y = np.random.randint(0, 2, n)  ❷
         y  ❷
Out[48]: array([1, 1, 0, 0, 1, 1, 0, 1, 0, 1])

In [49]: model = Sequential()
         model.add(Dense(256, activation='relu', input_dim=f))
         model.add(Dense(1, activation='sigmoid'))  ❸
         model.compile(loss='binary_crossentropy', optimizer='rmsprop',
                       metrics=['acc'])  ❹

In [50]: h = model.fit(x, y, epochs=50, verbose=False)
Out[50]: <keras.callbacks.callbacks.History at 0x7fde09dd1cd0>

In [51]: y_ = np.where(model.predict(x).flatten() > 0.5, 1, 0)
         y_
Out[51]: array([1, 1, 0, 0, 0, 1, 0, 1, 0, 1], dtype=int32)
```

```
In [52]: y == y_  ❺
Out[52]: array([ True,  True,  True,  True, False,  True,  True,  True,  True,
                 True])

In [53]: res = pd.DataFrame(h.history)  ❻

In [54]: res.plot(figsize=(10, 6));  ❻
```

❶ Creates random features data

❷ Creates random labels data

❸ Defines the activation function for the output layer as `sigmoid`

❹ Defines the loss function to be `binary_crossentropy`[6]

❺ Compares the predicted values with the labels data

❻ Plots the loss function and accuracy values for every training step

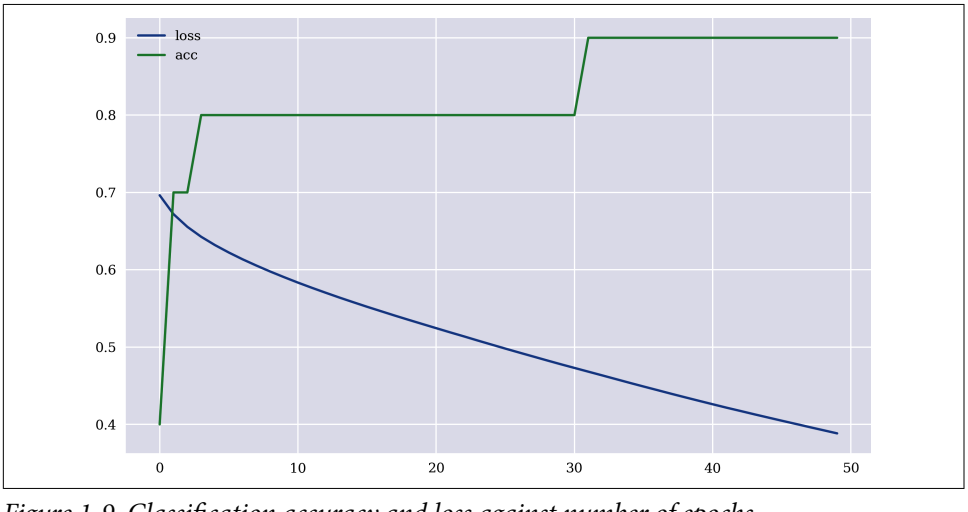


Figure 1-9. Classification accuracy and loss against number of epochs

6 The *loss function* calculates the prediction error of the neural network (or other ML algorithms). *Binary cross entropy* is an appropriate loss function for binary classification problems, while the *mean squared error* (MSE) is, for example, appropriate for estimation problems. For details on loss functions with `Keras`, see *https://keras.io/losses*.

The examples in this section illustrate some fundamental characteristics of neural networks as compared to OLS regression:

Problem-agnostic
: The neural network approach is agnostic when it comes to estimating and classifying label values, given a set of feature values. Statistical methods, such as OLS regression, might perform well for a smaller set of problems, but not too well or not at all for others.

Incremental learning
: The optimal weights within a neural network, given a target measure of success, are learned incrementally based on a random initialization and incremental improvements. These incremental improvements are achieved by considering the differences between the predicted values and the sample label values and back-propagating weights updates through the neural network.

Universal approximation
: There are strong mathematical theorems showing that neural networks (even with one hidden layer only) can approximate almost any function.[7]

These characteristics might justify why this book puts neural networks at the core with regard to the algorithms used. Chapter 2 discusses more good reasons.



Neural Networks

Neural networks are good at learning relationships between input and output data. They can be applied to a number of problem types, such as estimation in the presence of complex relationships or classification, for which traditional statistical methods are not well suited.

Importance of Data

The example at the end of the previous section shows that neural networks are capable of solving classification problems quite well. The neural network with one hidden layer reaches a high degree of accuracy on the given data set, or *in-sample*. However, what about the predictive power of a neural network? This hinges significantly on the volume and variety of the data available to train the neural network. Another numerical example, based on larger data sets, will illustrate this point.

7 See, for example, Kratsios (2019).

Small Data Set

Consider a random sample data set similar to the one used before in the classification example, but with more features and more samples. Most algorithms used in AI are about *pattern recognition*. In the following Python code, the number of binary features defines the number of possible patterns about which the algorithm can learn something. Given that the labels data is also binary, the algorithm tries to learn whether a `0` or `1` is more likely given a certain pattern, say `[0, 0, 1, 1, 1, 1, 0, 0, 0, 0]`. Because all numbers are randomly chosen with equal probability, there is not that much to learn beyond the fact that the labels `0` and `1` are equally likely no matter what (random) pattern is observed. Therefore, a baseline prediction algorithm should be accurate about 50% of the time, no matter what (random) pattern it is presented with:

```
In [55]: f = 10
         n = 250

In [56]: np.random.seed(100)

In [57]: x = np.random.randint(0, 2, (n, f))  ❶
         x[:4]  ❶
Out[57]: array([[0, 0, 1, 1, 1, 1, 0, 0, 0, 0],
                [0, 1, 0, 0, 0, 0, 1, 0, 0, 1],
                [0, 1, 0, 0, 0, 1, 1, 1, 0, 0],
                [1, 0, 0, 1, 1, 1, 1, 1, 0, 0]])

In [58]: y = np.random.randint(0, 2, n)  ❷
         y[:4]  ❷
Out[58]: array([0, 1, 0, 0])

In [59]: 2 ** f  ❸
Out[59]: 1024
```

❶ Features data

❷ Labels data

❸ Number of patterns

In order to proceed, the raw data is put into a `pandas` `DataFrame` object, which simplifies certain operations and analyses:

```
In [60]: fcols = [f'f{_}' for _ in range(f)]  ❶
         fcols  ❶
Out[60]: ['f0', 'f1', 'f2', 'f3', 'f4', 'f5', 'f6', 'f7', 'f8', 'f9']

In [61]: data = pd.DataFrame(x, columns=fcols)  ❷
         data['l'] = y  ❸
```

```
In [62]: data.info()  ❹
         <class 'pandas.core.frame.DataFrame'>
         RangeIndex: 250 entries, 0 to 249
         Data columns (total 11 columns):
          #   Column  Non-Null Count  Dtype
         ---  ------  --------------  -----
          0   f0      250 non-null    int64
          1   f1      250 non-null    int64
          2   f2      250 non-null    int64
          3   f3      250 non-null    int64
          4   f4      250 non-null    int64
          5   f5      250 non-null    int64
          6   f6      250 non-null    int64
          7   f7      250 non-null    int64
          8   f8      250 non-null    int64
          9   f9      250 non-null    int64
          10  l       250 non-null    int64
         dtypes: int64(11)
         memory usage: 21.6 KB
```

❶ Defines column names for the features data

❷ Puts the features data into a `DataFrame` object

❸ Puts the labels data into the same `DataFrame` object

❹ Shows the meta information for the data set

Two major problems can be identified given the results from executing the following Python code. First, not all patterns are in the sample data set. Second, the sample size is much too small per observed pattern. Even without digging deeper, it is clear that no classification algorithm can really learn about all the possible patterns in a meaningful way:

```
In [63]: grouped = data.groupby(list(data.columns))  ❶

In [64]: freq = grouped['l'].size().unstack(fill_value=0)  ❷

In [65]: freq['sum'] = freq[0] + freq[1]  ❸

In [66]: freq.head(10)  ❹
Out[66]: l                          0  1  sum
         f0 f1 f2 f3 f4 f5 f6 f7 f8 f9
         0  0  0  0  0  0  0  1  1  1   0  1    1
                           1  0  1  0   1  1    2
                                    1   0  1    1
                        1  0  0  0  0   1  0    1
                                    1   0  1    1
                              1  1  1   0  1    1
                           1  0  0  0   0  1    1
```

```
                        1  0   0  1    1
                  1  0  0  0  1  1   1  0    1
                        1  1  0  0   1  0    1
```

```
In [67]: freq['sum'].describe().astype(int)  ❺
Out[67]: count    227
         mean       1
         std        0
         min        1
         25%        1
         50%        1
         75%        1
         max        2
         Name: sum, dtype: int64
```

❶ Groups the data along all columns

❷ Unstacks the grouped data for the labels column

❸ Adds up the frequency for a `0` and a `1`

❹ Shows the frequencies for a `0` and a `1` given a certain pattern

❺ Provides statistics for the sum of the frequencies

The following Python code uses the `MLPClassifier` model from `scikit-learn`.[8] The model is trained on the whole data set. What about the ability of a neural network to learn about the relationships within a given data set? The ability is pretty high, as the in-sample accuracy score shows. It is in fact close to 100%, a result driven to a large extent by the relatively high neural network capacity given the relatively small data set:

```
In [68]: from sklearn.neural_network import MLPClassifier
         from sklearn.metrics import accuracy_score

In [69]: model = MLPClassifier(hidden_layer_sizes=[128, 128, 128],
                               max_iter=1000, random_state=100)

In [70]: model.fit(data[fcols], data['l'])
Out[70]: MLPClassifier(hidden_layer_sizes=[128, 128, 128], max_iter=1000,
                       random_state=100)

In [71]: accuracy_score(data['l'], model.predict(data[fcols]))
Out[71]: 0.952
```

8 For details, see `sklearn.neural_network.MLPClassifier`.

But what about the *predictive power* of a trained neural network? To this end, the given data set can be split into a training and a test data sub-set. The model is trained on the training data sub-set only and then tested with regard to its predictive power on the test data set. As before, the accuracy of the trained neural network is pretty high in-sample (that is, on the training data set). However, it is more than 10 percentage points worse than an uninformed baseline algorithm on the test data set:

```
In [72]: split = int(len(data) * 0.7)  ❶

In [73]: train = data[:split]  ❶
         test = data[split:]  ❶

In [74]: model.fit(train[fcols], train['l'])  ❷
Out[74]: MLPClassifier(hidden_layer_sizes=[128, 128, 128], max_iter=1000,
                       random_state=100)

In [75]: accuracy_score(train['l'], model.predict(train[fcols]))  ❸
Out[75]: 0.9714285714285714

In [76]: accuracy_score(test['l'], model.predict(test[fcols]))  ❹
Out[76]: 0.38666666666666666
```

❶ Splits the data into `train` and `test` data sub-sets

❷ Trains the model on the training data set only

❸ Reports the accuracy in-sample (training data set)

❹ Reports the accuracy out-of-sample (test data set)

Roughly speaking, the neural network, trained on a small data set only, learns wrong relationships due to the identified two major problem areas. The problems are not really relevant in the context of learning relationships *in-sample*. To the contrary, the smaller a data set is, the more easily in-sample relationships can be learned in general. However, the problem areas are highly relevant when using the trained neural network to generate predictions *out-of-sample*.

Larger Data Set

Fortunately, there is often a clear way out of this problematic situation: a *larger data set*. Faced with real-world problems, this theoretical insight might be equally correct. From a practical point of view, though, such larger data sets are not always available, nor can they often be generated so easily. However, in the context of the example of this section, a larger data set is indeed easily created.

The following Python code increases the number of samples in the initial sample data set significantly. The result is that the prediction accuracy of the trained neural

network increases by more than 10 percentage points, to a level of about 50%, which is to be expected given the nature of the labels data. It is now in line with an uninformed baseline algorithm:

```
In [77]: factor = 50

In [78]: big = pd.DataFrame(np.random.randint(0, 2, (factor * n, f)),
                            columns=fcols)

In [79]: big['l'] = np.random.randint(0, 2, factor * n)

In [80]: train = big[:split]
         test = big[split:]

In [81]: model.fit(train[fcols], train['l'])
Out[81]: MLPClassifier(hidden_layer_sizes=[128, 128, 128], max_iter=1000,
                       random_state=100)

In [82]: accuracy_score(train['l'], model.predict(train[fcols]))  ❶
Out[82]: 0.9657142857142857

In [83]: accuracy_score(test['l'], model.predict(test[fcols]))  ❷
Out[83]: 0.5043407707910751
```

❶ Prediction accuracy in-sample (training data set)

❷ Prediction accuracy out-of-sample (test data set)

A quick analysis of the available data, as shown next, explains the increase in the prediction accuracy. First, all possible patterns are now represented in the data set. Second, all patterns have an average frequency of above 10 in the data set. In other words, the neural network sees basically *all the patterns multiple times*. This allows the neural network to "learn" that both labels `0` and `1` are equally likely for all possible patterns. Of course, it is a rather involved way of learning this, but it is a good illustration of the fact that a *relatively small* data set might often be *too small* in the context of neural networks:

```
In [84]: grouped = big.groupby(list(data.columns))

In [85]: freq = grouped['l'].size().unstack(fill_value=0)

In [86]: freq['sum'] = freq[0] + freq[1]  ❶

In [87]: freq.head(6)
Out[87]: l                          0  1  sum
         f0 f1 f2 f3 f4 f5 f6 f7 f8 f9
         0  0  0  0  0  0  0  0  0  0   10  9   19
                                       1   5  4    9
                                    1  0   2  5    7
                                       1   6  6   12
```

```
                          1  0  0    9  8   17
                             1    7  4   11

In [88]: freq['sum'].describe().astype(int)  ❷
Out[88]: count    1024
         mean       12
         std         3
         min         2
         25%        10
         50%        12
         75%        15
         max        26
         Name: sum, dtype: int64
```

❶ Adds the frequency for the `0` and `1` values

❷ Shows summary statistics for the sum values



Volume and Variety

In the context of neural networks that perform prediction tasks, the volume and variety of the available data used to train the neural network are decisive for its prediction performance. The numerical, hypothetical examples in this section show that the same neural network trained on a relatively small and not-as-varied data set underperforms its counterpart trained on a relatively large and varied data set by more than 10 percentage points. This difference can be considered huge given that AI practitioners and companies often fight for improvements as small as a tenth of a percentage point.

Big Data

What is the difference between a *larger* data set and a *big* data set? The term *big data* has been used for more than a decade now to mean a number of things. For the purposes of this book, one might say that a *big data set* is large enough—in terms of volume, variety, and also maybe velocity—for an AI algorithm to be trained properly such that the algorithm performs better at a prediction task as compared to a baseline algorithm.

The larger data set used before is still small in practical terms. However, it is large enough to accomplish the specified goal. The required volume and variety of the data set are mainly driven by the structure and characteristics of the features and labels data.

In this context, assume that a retail bank implements a neural network–based classification approach for credit scoring. Given in-house data, the responsible data scientist